

Documentazione Progetto InfoLab

Giovanni Menicucci

June 2026

1 Introduzione

Questo documento tiene traccia dello sviluppo del progetto finale per il corso di Informatica con Laboratorio 2025/26, riguardante la soluzione dell'equazione del calore 2D.

2 Struttura del progetto

I task richiesti nel progetto vengono eseguiti con codici C++ (reperibili nella cartella "codice.cpp" o Python (nella cartella "codice.python"). Alcuni codici producono file di testo (nella cartella "dati"), che poi vengono letti dai codici successivi. Tutti i task sono ripetuti per $N = 32, 64, 128, 256, 512, 1024$ (e anche per $N = 4$ che è verificabile a mano), e se necessario per i due ordinamenti definiti nel testo.

Le sezioni seguenti contengono i moduli principali che compongono il progetto.

3 Discretizzazione del dominio

Input: numero naturale N . Il dominio $[0, 1]^2$ è discretizzato con una griglia equispaziata di $(N + 2) \times (N + 2)$ punti.

Output: grafo di adiacenza, i cui nodi sono i punti interni alla griglia, con archi tra nodi primi vicini nella griglia.

File coinvolti: codice `discretizzazione.cpp` che genera un file `coords.txt` che rappresenta la griglia e un file `connectivity.txt` che rappresenta il grafo.

Strutture dati necessarie:

- **struct** per i nodi, per memorizzare indice progressivo n , indici della griglia i, j , coordinate spaziali x, y .
- **vector** per contenere la sequenza dei nodi interni, ordinati secondo l'indice n , per poi esportarli su `coords.txt`.
- **vector** di **vector** per la lista di adiacenza del grafo (lista e non matrice perché il grafo è sparso); e anche per costruire una matrice **griglia** che associ alle coordinate spaziali di ciascun nodo interno il relativo indice progressivo, e associ -1 ai nodi di bordo.

Funzioni:

- **GeneraGriglia.** Calcola il passo della griglia e genera i singoli nodi interni con un doppio ciclo for che assegna indice progressivo n (ordine lessicografico), coordinate nella griglia e spaziali. Riempie il vettore dei nodi interni e la matrice `griglia`, aumentando n di 1 alla fine di ogni iterazione. La matrice è inizializzata nel `main` con valori -1 . Complessità: $O(N^2)$, a causa del doppio ciclo for. All'interno di ogni iterazione si fanno solo operazioni elementari.
- **EsportaPuntiInterni.** Crea il file `coordsN.txt`, scorre sul vettore dei nodi interni e li scrive sul file, usando una riga per ciascun nodo, contenente n, i, j, x, y . Complessità: $O(N^2)$ (lunghezza del vettore su cui scorro).
- **GeneraEsportaGrafo.** Crea il file `connectivityN.txt`. Svolge un doppio ciclo for, sugli indici dei nodi interni. Ad ogni iterazione, usa la matrice `griglia` per verificare se i 2 vicini a destra e in alto del nodo corrente sono di bordo o no (se un nodo è di bordo, il valore assegnatogli nella matrice è rimasto -1 , cioè non è stato sovrascritto con l'indice progressivo). Controlla solo i vicini a destra e in alto, perché se controllassi tutti e 4 conterei due volte gli archi. Se il vicino non è di bordo, aggiunge alla lista di adiacenza (vector di N vector, inizializzato nel `main`) i due nodi coinvolti. Poi scrive in una riga del file l'indice progressivo e dell'arco (cioè quanti archi ho creato fino a questo) e gli indici progressivi dei due nodi. Complessità: $O(N^2)$, a causa del doppio ciclo for.

Complessità totale: $O(N^2)$.

4 Riordinamento dei nodi

Input: grafo definito sopra.

Output: nodi del grafo riordinati.

File coinvolti: codice `riordinamento.cpp` che prende il file `coords.txt` generato nel modulo precedente, e genera un file `ordering.txt` che rappresenta la permutazione dei nodi.

Strutture dati necessarie:

- `struct` per i nodi, come nel modulo di discretizzazione.
- `vector` per le sequenze di nodi.

Funzioni:

- **GeneraVettoreNodi.** Prende in input N e un vettore vuoto definito nel `main`. Scorre sul file `coordsN.txt` e legge gli indici n, i, j, x, y dei nodi, riempiendo il vettore con i nodi. In pratica inverte quanto fatto nella funzione `EsportaPuntiInterni` del modulo di discretizzazione. Complessità: $O(N^2)$ che è il numero di nodi.
- **GeneraNuovoOrdinamento.** Prende in input il vettore dei nodi riempito dalla funzione precedente, un vettore vuoto che conterrà i nodi riordinati, e gli indici i, j massimo e minimo dei nodi contenuti nel rettangolo da riordinare (quando la chiamo dal `main`, sarà $i_{min} = j_{min} = 1$, $i_{max} = j_{max} = N$, cioè tutta la griglia dei nodi interni). Calcola le dimensioni N_1, N_2 del rettangolo da dividere in tre. Il caso base della ricorsione è quando il rettangolo non è divisibile ulteriormente, cioè contiene un solo nodo (oppure non è nemmeno definito perché $i_{min} > i_{max}$, può succedere). Se si supera il caso base, si divide il rettangolo verticalmente o orizzontalmente, in base a quale lato del rettangolo è maggiore. Si prende la riga o colonna di mezzo come separatore e si scorre sul vettore dei nodi, assegnandoli ai tre vettori V_1, V_2, V_S definiti nel testo. Poi applica la funzione a V_1 e V_2 (passo ricorsivo), facendo attenzione agli indici che li delimitano. Infine aggiunge il vettore V_S in fondo al vettore dei nodi riordinati; gli altri nodi (che non fossero nei casi base) sono stati aggiunti da questa stessa riga di codice durante la ricorsione.

Complessità: $O(N^2 \log N)$, perché ad ogni passo della ricorsione devo scorrere sul vettore dei nodi corrente, che ha lunghezza $O(N^2)$, e ad ogni passo della ricorsione divido per 2 (circa) il lato del rettangolo considerato, quindi il numero di passi è $O(\log N)$.

- **EsportaOrdinamento.** Crea il file `ordering.txt` e scorre sul vettore dei nodi riordinati, generato dalla funzione sopra. Per ciascun nodo, scrive in una riga del file la posizione m nel nuovo ordine e l'indice progressivo iniziale n contenuto nella struct. Complessità: $O(N^2)$, che è la lunghezza del vettore.

Complessità totale: $O(N^2 \log N)$.

5 Approssimazione dell'equazione differenziale con sistema lineare

Input: dominio dell'equazione differenziale discretizzato (ordinato nei due modi), coefficienti, funzione sorgente, condizione al bordo.

Output: matrice e termine noto del sistema lineare che rappresenta l'equazione.

File coinvolti: codice `approx_lineare.cpp` che prende i file `coords.txt` (generato nel modulo "discretizzazione") e `ordering.txt` (generato nel modulo "riordinamento") e genera un file `A.txt` che rappresenta la matrice del sistema lineare, e un file `rhs.txt` che rappresenta il termine noto.

Strutture dati necessarie:

- `vector` per matrici e vettori.
- `pair` per coppie di indici.

Funzioni:

- **GeneraIndiciBordo.** Per accettare in input una condizione al bordo non identicamente nulla, assegno a ogni punto del bordo della griglia un indice progressivo tra -1 e $-4N - 4$, cioè creo una funzione biunivoca $p(i, j)$ tra gli indici (i, j) di bordo e gli interi $\{-1, \dots, -4N - 4\}$. Lo faccio scorrendo in senso orario sul bordo, partendo dall'origine. Assegno ai punti del bordo indici negativi per poter distinguere facilmente punti di bordo e non. Se nella funzione successiva **GeneraEsportaMatriceRhs** si vuole prendere in input una condizione al bordo, essa dovrà essere un vettore v di $4N + 4$ numeri reali, ordinati nel modo descritto sopra, in modo che $u_0(x_i, y_j) = v[-p(i, j) - 1]$.

Complessità: $O(N)$, che è la lunghezza del bordo.

- **GeneraMappaNaturale.** Genera una mappa da (i, j) a n (ordine naturale), e la sua inversa. Legge il file `coords.txt` e scrive su una matrice $(N + 2) \times (N + 2)$ (la stessa usata per i punti di bordo) l'indice progressivo n associato a ogni valore di (i, j) interno. Inoltre riempie un vettore che fa l'inverso, cioè associa la coppia (i, j) a n (solo per i punti interni). Infine chiama la funzione **GeneraIndiciBordo** per associare i valori da -1 a $-4N - 4$ ai punti di bordo.

Complessità: $O(N^2)$, il numero di elementi nella matrice..

- **GeneraMappaOrdinata.** Genera una mappa da (i, j) a m (riordinamento), e la sua inversa. Legge il file `ordering.txt` e fa l'analogo della funzione precedente, usando la mappa definita sopra per passare da (i, j) a m attraverso n (poiché il file txt contiene la coppia m, n).

Complessità: $O(N^2)$, come sopra.

- **GeneraEsportaMatriceRhs.** Genera i coefficienti non nulli della matrice A del sistema ($N^2 \times N^2$), il termine noto, e li esporta. Prende in input:

1. N .
2. Una stringa "nat" o "ord" che decide il tipo di ordinamento (per dare il nome giusto ai file).
3. La matrice e il vettore generati dalle funzioni sopra (devono essere quelli relativi all'ordinamento scelto).
4. La funzione che fa da sorgente di calore, definita nel main come funzione lambda. Per darla come parametro, serve includere `functional`.
5. La diffusività κ , che di default è 0.01.
6. Un vettore di $4N + 4$ componenti che contiene le condizioni al bordo $u_0(x, y)$. Di default la condizione di bordo dev'essere 0, ma non posso mettere come valore di default un vettore la cui lunghezza dipenda da N , che è un argomento della funzione. Perciò di default il vettore dei valori al bordo è vuoto, e in tal caso lo ridefinisco all'interno della funzione in modo che contenga solo zeri.

La funzione crea un file `A.txt` e un file `rhs.txt` (ne servirà uno per ogni N e per i due ordinamenti).

Scorre sugli indici di riga l di A , cioè sulle N^2 righe del sistema lineare. Usando una delle mappe definite sopra, associa a l una coppia (i, j) . Calcola il valore $f(x_i, y_j)$ e lo salva in una variabile b , che sarà il coefficiente l del termine noto. Dall'equazione scritta nel testo si vede che solo 5 coefficienti ($(i + 1, j)$ etc) sono coinvolti nella riga (i, j) del sistema. Usando una delle mappe definite sopra, associa a quelli che non sono di bordo (cioè hanno immagine non negativa tramite la mappa; sono almeno 3 e al massimo 5) un indice m . Calcola il valore di $A_{l,m}$ e scrive sul file `A.txt` la riga $l, m, A_{l,m}$. Il formato è COO, cioè sto salvando indici e valore solo dei coefficienti non nulli.

Per gli eventuali indici di bordo, usa l'indice m calcolato con la mappa per prendere il valore $u_0(x, y)$ assegnato, e lo somma alla variabile b (con opportuno segno e coefficiente). Infine scrive sul file `rhs.txt` il valore di b .

Complessità: $O(N^2)$, perché il ciclo su l fa N^2 iterazioni, e ogni iterazione contiene poche operazioni che costano $O(1)$ (il calcolo della funzione $f(x, y)$ potrebbe essere pesante, ma non dipende da N).

Complessità totale: $O(N^2)$.

6 Fattorizzazione di Cholesky

Input: matrice A del sistema lineare.

Output: matrice L tale che $A = -LL^T$. Immagini della struttura di A, L (entrate non nulle).

File coinvolti: notebook `soluzione_sistema.ipynb` che legge `A.txt` generato nel modulo precedente ed esegue la fattorizzazione. Struttura della matrice (entrate non nulle).

Strutture dati:

- array di NumPy, per salvare A in formato COO.
- Formato CSC fornito da SciPy.

Funzioni:

- `ImportaMatriceCSC`. Prende in input N e una stringa "ord" o "nat" per scegliere il file. Legge `A.txt` in formato COO, scarica le tre colonne come array (con `np.loadtxt`), e converte in CSC (con `scipy.sparse.csc_matrix`). Restituisce la matrice A in CSC. Complessità: $O(N^2)$, che è il numero di entrate non nulle in A .
- `CalcolaCholesky`. Prende in input una matrice A (definita negativa) e calcola L (tale che $LL^T = -A$) con la funzione fornita nel testo.

7 Soluzione del sistema lineare

Input: fattorizzazione di A e termine noto del sistema lineare.

Output: soluzione del sistema lineare. Analisi del tempo necessario al variare di N e dell'ordinamento dei nodi. Plot della soluzione.

File coinvolti: notebook `soluzione_sistema.ipynb` che legge il file `rhs.txt` generato nel modulo "approssimazione".

Strutture dati:

- array di NumPy per i vettori del termine noto e della soluzione, e per le matrici necessarie a passare da n o m a (i, j) .

Funzioni:

- **ImportaRhs.** Legge `rhs.txt` e scarica il termine noto come array b . Complessità: $O(N^2)$, che è il numero di coefficienti di b .
- **RisolviSistema.** Prende in input L triangolare inferiore, e b array. Risolve prima il sistema triangolare inferiore $Ly = -b$, poi quello triangolare superiore $L^T u = y$, con la funzione `scipy.sparse.linalg.spsolve_triangular`. L'output è il vettore u , nell'ordine scelto. Complessità: $O(nnz(L))$.
- **GeneraMappaNaturale.** Legge il file `coords.txt` e genera un array che in posizione n (indice progressivo naturale) contiene la coppia (i, j) associata. Complessità: $O(N^2)$, la lunghezza dell'array.
- **GeneraMappaOrdinata.** Legge il file `ordering.txt` e genera un array che in posizione m (indice progressivo dopo il riordinamento) contiene la coppia (i, j) associata. Per convertire da m a (i, j) passo attraverso n , usando la funzione precedente. Uso il modo nativo di NumPy per permutare gli elementi ("fancy indexing"): valuto l'array "mappa naturale" sull'intero array di conversione tra gli ordinamenti m e n . Il risultato è l'array voluto che mappa m in (i, j) . Complessità: $O(N^2)$, perché scaricare dal file, chiamare la funzione precedente e permutare i coefficienti richiedono tutti $O(N^2)$.

Queste due funzioni sono analoghe alle funzioni **GeneraMappaNaturale/Ordinata** del modulo "approssimazione lineare", ma operano solo in un verso (da n o m a (i, j)).

- **GeneraMappaBordo.** Genera un array di $4N + 4$ elementi che in posizione k contiene il $k + 1$ -esimo punto (i, j) che si trova scorrendo in senso orario sul bordo del quadrato, partendo dall'origine. Per farlo uso gli array di NumPy, che sono più veloci dei cicli `for` di Python. Complessità: $O(N)$, che è la lunghezza del bordo.

Questa funzione fa l'operazione inversa della funzione **GeneraIndiciBordo** del modulo "approssimazione lineare", a meno del segno che lì era stato usato per distinguere tra punti di bordo e non.

- **DisegnaSoluzione.** Prende in input N , una stringa che specifica l'ordinamento, il vettore soluzione u in quell'ordinamento e il vettore delle condizioni al bordo nel formato definito nel modulo "approssimazione lineare". Usa le funzioni definite sopra per trasformare dall'ordinamento scelto agli indici (i, j) , e disegna la soluzione.